

Módulo: Comandos de Atualização e Remoção de Dados (UPDATE e DELETE)

Curso: Banco de Dados Relacionais aplicado à Administração Tributária **Público-alvo:** Profissionais da Secretaria de Estado da Fazenda da Paraíba (SEFAZ-PB)

SGBD utilizado: Microsoft SQL Server (T-SQL) — ferramenta: SQL Server

Management Studio (SSMS) **Carga horária sugerida:** 8 horas (4h teoria + 4h prática)

Sumário

1. [Objetivos do Módulo](#)
 2. [UPDATE e DELETE no Contexto da Linguagem SQL](#)
 3. [O Comando UPDATE — Sintaxe Completa](#)
 4. [O Comando DELETE — Sintaxe Completa](#)
 5. [TRUNCATE TABLE × DELETE — Diferenças Fundamentais](#)
 6. [Interação com Integridade Referencial e Triggers](#)
 7. [Segurança Operacional: Evitando Desastres em Produção](#)
 8. [Roteiro Prático Completo](#)
 9. [Exercícios Práticos](#)
 10. [Glossário](#)
 11. [Referências](#)
-

1. Objetivos do Módulo

Ao final deste módulo, o participante será capaz de:

- Escrever comandos **UPDATE** em todas as suas variações de sintaxe em T-SQL: com condições simples, com **JOIN** (cláusula **FROM**), com subconsultas correlacionadas, com **CASE**, com **TOP**, com **OUTPUT** e em construções baseadas em CTE.

- Escrever comandos **DELETE** em todas as suas variações: com condições simples, com **JOIN**, com subconsultas, com **TOP**, com **OUTPUT** e em padrões de exclusão em lote (*batched delete*).
 - Diferenciar **DELETE** de **TRUNCATE TABLE**, sabendo exatamente quando cada um é apropriado.
 - Prever e tratar o impacto de **UPDATE/DELETE** sobre chaves estrangeiras, **CASCADE**, **SET NULL** e *triggers* já configurados no banco.
 - Aplicar práticas de segurança operacional que evitam a alteração ou exclusão acidental de grandes volumes de dados fiscais.
-

2. UPDATE e DELETE no Contexto da Linguagem SQL

UPDATE e **DELETE**, junto com **INSERT** e **SELECT**, compõem o núcleo da **DML (Data Manipulation Language)** — o subconjunto do SQL responsável por manipular os **dados** armazenados nas tabelas (em contraste com a **DDL**, vista no módulo de **ALTER TABLE**, que manipula a **estrutura** do banco).

Por que este módulo merece atenção redobrada em um contexto fiscal:

diferentemente de um **SELECT** (que apenas lê dados, sem risco de alterá-los), **UPDATE** e **DELETE** **modificam permanentemente** o estado da base — e, em uma base fiscal, isso pode significar alterar o valor de uma multa já notificada, ou excluir um documento fiscal que tem valor probatório. Um **UPDATE** ou **DELETE** mal escrito (por exemplo, sem uma cláusula **WHERE**) pode afetar **milhões de registros em uma fração de segundo**, e nem sempre é possível reverter o dano. Este módulo trata tanto da sintaxe completa quanto da disciplina operacional necessária para usá-la com segurança.

3. O Comando UPDATE — Sintaxe Completa

3.1 Sintaxe Básica: UPDATE ... SET ... WHERE

A forma mais simples do comando altera o valor de uma ou mais colunas, para todas as linhas que satisfazem uma condição.

```
UPDATE tabela
SET coluna1 = valor1
WHERE condicao;
```

Exemplo fiscal: marcar uma NF-e específica como cancelada.

```
UPDATE nfe
SET situacao_nfe = 'CANCELADA'
WHERE id_nfe = 1042;
```

A cláusula **WHERE** não é obrigatória sintaticamente — mas é, na prática, a parte mais importante do comando. Um **UPDATE** sem **WHERE** altera **todas** as linhas da tabela. Veremos essa armadilha com mais detalhe na seção 7.

3.2 Atualizando Múltiplas Colunas

Basta separar cada atribuição por vírgula dentro da cláusula **SET**.

Exemplo fiscal: ao cancelar uma NF-e, registrar também a data e o motivo do cancelamento.

```
UPDATE nfe
SET
    situacao_nfe          = 'CANCELADA',
    data_cancelamento    = SYSDATETIME(),
    motivo_cancelamento = 'Erro de digitação no CNPJ do destinatário'
WHERE id_nfe = 1042;
```

3.3 Atualizando com Base em Expressões (referenciando o próprio valor atual)

O lado direito do **SET** pode ser uma **expressão** que referencia o valor atual da própria coluna (ou de outras colunas da mesma linha) — o SQL Server sempre calcula a expressão com base no valor **anterior** à atualização.

Exemplo fiscal: reajustar em 10% o valor de todas as multas aplicadas ainda não quitadas, por correção monetária.

```
UPDATE auto_infracao
SET valor_multa_aplicado = valor_multa_aplicado * 1.10
WHERE situacao = 'LAVRADO';
```

3.4 Atualizando com Expressões Condicionais (CASE)

A expressão **CASE** permite atribuir valores diferentes a depender de uma condição, tudo em um único comando.

Exemplo fiscal: reclassificar automaticamente a situação cadastral de contribuintes com base no tempo desde a última NF-e emitida.

```
UPDATE contribuinte
SET situacao_cadastral =
CASE
    WHEN DATEDIFF(MONTH, (SELECT MAX(n.data_emissao)
                           FROM nfe n
                           WHERE n.id_contribuinte_emitente =
contribuinte.id_contribuinte), GETDATE()) > 24
    THEN 'INAPTO'
    WHEN DATEDIFF(MONTH, (SELECT MAX(n.data_emissao)
                           FROM nfe n
                           WHERE n.id_contribuinte_emitente =
contribuinte.id_contribuinte), GETDATE()) > 12
    THEN 'SUSPENSO'
    ELSE situacao_cadastral -- mantém o valor atual nos demais casos
END
WHERE situacao_cadastral = 'ATIVO';
```

Repare no **ELSE situacao_cadastral**: é uma técnica comum para "não alterar" a coluna quando nenhuma das condições específicas se aplica — a expressão **CASE** sempre precisa produzir algum valor, e reatribuir o valor atual equivale, na prática, a "não mudar nada" para aquela linha.

3.5 UPDATE ... FROM — Atualizando com Base em Outra Tabela (JOIN)

Esta é uma das construções mais usadas — e mais específicas do T-SQL — para atualizar uma tabela **com base em valores de outra tabela relacionada**. A cláusula **FROM**, dentro de um **UPDATE**, introduz o **JOIN**.

Exemplo fiscal: uma nova portaria da SEFAZ-PB reajusta o valor-base de multa de cada tipo de infração em 15%. É necessário recalcular o `valor_multa_aplicado` de todos os autos de infração **ainda não quitados**, com base no **novo** `valor_multa_base` de seu respectivo tipo de infração.

```
UPDATE a
SET a.valor_multa_aplicado = t.valor_multa_base
FROM auto_infracao a
JOIN tipo_infracao t
    ON t.id_tipo_infracao = a.id_tipo_infracao
WHERE a.situacao = 'LAVRADO';
```

Atenção à sintaxe: o alias `a` (definido na cláusula `FROM`) é usado tanto no `UPDATE a` quanto no `SET a.coluna = ...` — essa é a forma padrão do T-SQL. Note que a tabela mencionada logo após `UPDATE` deve corresponder à tabela (ou ao seu alias) que você realmente pretende alterar; é um erro comum, especialmente para quem vem de outros SGBDs, esquecer que o T-SQL permite (e exige, neste padrão) repetir o alias logo após `UPDATE`.

3.6 UPDATE com Subconsulta Correlacionada

Quando a lógica de atualização depende de uma verificação de existência (ou de um valor agregado) em outra tabela, mas não se encaixa bem em um `JOIN` direto, usa-se uma subconsulta no `WHERE` ou no `SET`.

Exemplo fiscal: marcar como `'EM_MALHA_FISCAL'` todo contribuinte que possua ao menos uma NF-e autorizada sem o correspondente registro na EFD do mesmo período (situação já estudada nos módulos anteriores).

```
UPDATE contribuinte
SET situacao_cadastral = 'EM_MALHA_FISCAL'
WHERE EXISTS (
    SELECT 1
    FROM nfe n
    WHERE n.id_contribuinte_emitente = contribuinte.id_contribuinte
        AND n.situacao_nfe = 'AUTORIZADA'
        AND NOT EXISTS (
            SELECT 1
            FROM efd_registro_c100 c
            WHERE c.chave_acesso_nfe = n.chave_acesso
        )
);
```

3.7 UPDATE TOP (n) — Limitando o Número de Linhas Afetadas

TOP (n) restringe a atualização às primeiras **n** linhas que satisfazem a condição — útil, sobretudo, para processar grandes volumes **em lotes**, como veremos na seção 7.

```
UPDATE TOP (500) auto_infracao
SET situacao = 'EM_RECURSO'
WHERE situacao = 'LAVRADO'
AND id_auto_infracao IN (SELECT id_auto_infracao FROM recurso_administrativo);
```

Cuidado: sem uma cláusula **ORDER BY** (que, tecnicamente, o **UPDATE** não aceita diretamente — apenas o **SELECT** aceita), **não há garantia de quais linhas específicas** serão as "primeiras 500" escolhidas pelo SQL Server. Para um controle determinístico de quais linhas processar em cada lote, é necessário combinar **TOP** com uma CTE ordenada, como veremos na seção 3.9.

3.8 UPDATE com a Cláusula OUTPUT — Capturando Valores Antes e Depois

A cláusula **OUTPUT** permite **capturar** as linhas afetadas por um **UPDATE** — tanto o valor **antigo** (**deleted.coluna**) quanto o valor **novo** (**inserted.coluna**) — sem precisar de uma consulta separada. É extremamente útil para trilhas de auditoria.

Exemplo fiscal: ao reajustar o valor de multas (seção 3.5), registrar uma trilha de auditoria com o valor antes e depois da alteração.

```
CREATE TABLE auditoria_reajuste_multa (
    id_auditoria          BIGINT IDENTITY(1,1) NOT NULL,
    id_auto_infracao      INT                NOT NULL,
    valor_anterior        NUMERIC(15,2) NOT NULL,
    valor_novo            NUMERIC(15,2) NOT NULL,
    data_hora_alteracao   DATETIME2         NOT NULL DEFAULT
(SYSDATETIME()),
    CONSTRAINT pk_auditoria_reajuste_multa PRIMARY KEY (id_auditoria)
);
GO

UPDATE a
SET a.valor_multa_aplicado = t.valor_multa_base
OUTPUT
    deleted.id_auto_infracao,
```

```

deleted.valor_multa_aplicado,      -- valor ANTES da atualização
inserted.valor_multa_aplicado      -- valor DEPOIS da atualização
INTO auditoria_reajuste_multa (id_auto_infracao, valor_anterior, valor_novo)
FROM auto_infracao a
JOIN tipo_infracao t ON t.id_tipo_infracao = a.id_tipo_infracao
WHERE a.situacao = 'LAVRADO';

```

deleted e inserted dentro do OUTPUT de um UPDATE: o nome pode causar estranheza à primeira vista, mas é o mesmo par de tabelas virtuais já estudado nas *triggers* (módulo de Restrições e Integridade): dentro de um **UPDATE**, o SQL Server trata a operação internamente como "excluir a linha antiga e inserir a linha nova" — por isso **deleted** representa o estado **antes**, e **inserted**, o estado **depois**.

3.9 UPDATE por meio de uma CTE (Common Table Expression)

Uma **CTE** (**WITH nome AS (...)**) pode ser usada para tornar a linha-alvo do **UPDATE** mais explícita — especialmente útil para combinar **TOP** com um critério de ordenação determinístico (o que o **UPDATE TOP** sozinho não permite, como visto na seção 3.7).

Exemplo fiscal: processar em lotes de 500 as NF-e mais antigas pendentes de vinculação ao registro EFD correspondente, evitando travar a tabela inteira de uma só vez.

```

WITH lote_a_atualizar AS (
    SELECT TOP (500) id_nfe, situacao_nfe
    FROM nfe
    WHERE situacao_nfe = 'PENDENTE_VINCULACAO'
    ORDER BY data_emissao ASC      -- as mais antigas primeiro – determinístico
)
UPDATE lote_a_atualizar
SET situacao_nfe = 'VINCULACAO_EM_PROCESSAMENTO';

```

A CTE aqui funciona como uma "janela" sobre a tabela **nfe**, definida por uma consulta com **ORDER BY** — algo que o **UPDATE TOP** direto não oferece. O **UPDATE** final é aplicado sobre essa janela, e o SQL Server entende que a alteração deve refletir na tabela base (**nfe**).

3.10 Resumo da Sintaxe Completa do UPDATE

```
[ WITH cte AS (...) ]  
UPDATE [ TOP (n) ] { tabela | cte | alias }  
SET coluna1 = expressao1 [, coluna2 = expressao2, ... ]  
[ OUTPUT clausula_output [ INTO tabela_destino ] ]  
[ FROM origem_do_join [ JOIN ... ON ... ] ]  
[ WHERE condicao ]
```

4. O Comando DELETE — Sintaxe Completa

4.1 Sintaxe Básica: DELETE FROM ... WHERE

```
DELETE FROM tabela  
WHERE condicao;
```

Exemplo fiscal: remover uma duplicata de NF-e inserida por engano (registro de teste).

```
DELETE FROM duplicata_nfe  
WHERE id_duplicata = 87;
```

Em T-SQL, a palavra **FROM** após **DELETE** é **opcional** na sintaxe básica (**DELETE tabela WHERE ...** também funciona), mas a forma **DELETE FROM tabela** é a mais recomendada por clareza e por ser exigida nas variações mais complexas (como a **DELETE ... FROM** com **JOIN**, seção 4.3, onde o **FROM** tem um papel sintático diferente).

4.2 DELETE com Subconsulta (IN, EXISTS, NOT EXISTS)

Exemplo fiscal — usando `IN`: remover, da tabela de staging, todas as linhas cuja `chave_acesso` já foi migrada com sucesso para a tabela definitiva (rotina de limpeza pós-carga, tema do módulo de `BULK INSERT`).

```
DELETE FROM stg_nfe_importacao
WHERE chave_acesso IN (SELECT chave_acesso FROM nfe_importada);
```

Exemplo fiscal — usando `NOT EXISTS`: remover eventos de NF-e (`evento_nfe`) órfãos, cuja NF-e correspondente foi excluída de um ambiente de testes (situação que não deveria ocorrer em produção graças à integridade referencial, mas é comum em bases de homologação sem `FOREIGN KEY` totalmente configurada).

```
DELETE FROM evento_nfe
WHERE NOT EXISTS (
    SELECT 1 FROM nfe n WHERE n.id_nfe = evento_nfe.id_nfe
);
```

4.3 DELETE ... FROM — Removendo com Base em um JOIN

Assim como o `UPDATE`, o `DELETE` pode usar uma cláusula `FROM` com `JOIN` para decidir quais linhas remover com base em outra tabela.

Exemplo fiscal: remover todos os itens (`item_nfe`) de notas fiscais que foram **denegadas** (documento fiscal que nunca chegou a ter validade jurídica, situação distinta de "cancelada").

```
DELETE i
FROM item_nfe i
JOIN nfe n ON n.id_nfe = i.id_nfe
WHERE n.situacao_nfe = 'DENEGADA';
```

Note a semelhança estrutural com o `UPDATE ... FROM` (seção 3.5): o alias (`i`) aparece logo após `DELETE`, indicando qual das tabelas envolvidas no `JOIN` deve, de fato, ter suas linhas removidas — a tabela `nfe` (`n`) é usada apenas como critério de filtro, e **suas linhas não são afetadas**.

4.4 DELETE TOP (n)

Assim como no **UPDATE**, é possível limitar o número de linhas removidas de uma vez.

```
DELETE TOP (1000) FROM stg_nfe_importacao
WHERE data_hora_carga < DATEADD(DAY, -90, GETDATE());
```

Vale a mesma ressalva da seção 3.7: sem **ORDER BY**, não há garantia de **quais** 1000 linhas serão excluídas primeiro — para exclusões em lote com ordem determinística, combine com uma CTE (seção 4.6).

4.5 DELETE com a Cláusula OUTPUT — Capturando os Dados Excluídos

Assim como no **UPDATE**, é possível capturar as linhas removidas — extremamente valioso quando a exclusão de dados fiscais exige, por si só, uma trilha de auditoria (ou um "backup lógico" antes do apagamento definitivo).

Exemplo fiscal: antes de expurgar registros antigos de staging (mais de 90 dias), arquivá-los em uma tabela de histórico, em vez de simplesmente perdê-los.

```
CREATE TABLE stg_nfe_importacao_arquivo_morto (
    chave_acesso          VARCHAR(44),
    cnpj_emitente         VARCHAR(14),
    cnpj_destinatario     VARCHAR(14),
    numero_nf             VARCHAR(20),
    serie                 VARCHAR(10),
    data_emissao          VARCHAR(10),
    valor_total           VARCHAR(20),
    situacao              VARCHAR(20),
    nome_arquivo_origem   VARCHAR(200),
    data_hora_carga       DATETIME2,
    data_hora_expurgo     DATETIME2 DEFAULT
    (SYSDATETIME())
);
GO

DELETE FROM stg_nfe_importacao
OUTPUT
    deleted.chave_acesso, deleted.cnpj_emitente, deleted.cnpj_destinatario,
    deleted.numero_nf, deleted.serie, deleted.data_emissao, deleted.valor_total,
    deleted.situacao, deleted.nome_arquivo_origem, deleted.data_hora_carga
INTO stg_nfe_importacao_arquivo_morto
(chave_acesso, cnpj_emitente, cnpj_destinatario, numero_nf, serie,
```

```
data_emissao, valor_total, situacao, nome_arquivo_origem, data_hora_carga)
WHERE data_hora_carga < DATEADD(DAY, -90, GETDATE());
```

Este padrão — **DELETE ... OUTPUT deleted.* INTO tabela_arquivo** — é, na prática, a forma correta de implementar um "soft delete com arquivamento" em T-SQL, sem precisar de duas instruções separadas (**INSERT** seguido de **DELETE**), o que evitaria uma condição de corrida entre as duas operações.

4.6 DELETE por meio de uma CTE

Da mesma forma que no **UPDATE**, uma CTE viabiliza a exclusão em lotes com ordem determinística.

Exemplo fiscal: expurgar, em lotes de 1000, os registros mais antigos de **stg_nfe_importacao_arquivo_morto** que já ultrapassaram o prazo legal de retenção (por exemplo, 5 anos).

```
WITH lote_a_excluir AS (
    SELECT TOP (1000) chave_acesso
    FROM stg_nfe_importacao_arquivo_morto
    WHERE data_hora_expurgo < DATEADD(YEAR, -5, GETDATE())
    ORDER BY data_hora_expurgo ASC
)
DELETE FROM lote_a_excluir;
```

4.7 Exclusão em Lote (Batched DELETE) para Tabelas Grandes

Excluir milhões de linhas em um único **DELETE** gera um **crescimento explosivo do log de transações** (cada linha excluída é registrada individualmente no log, dentro de uma única transação gigante) e mantém bloqueios por um tempo prolongado. A prática recomendada é **excluir em lotes menores, em um laço**, permitindo que cada lote seja committado (e o espaço de log, potencialmente reutilizado) antes do próximo.

```
DECLARE @linhas_excluidas INT = 1;

WHILE @linhas_excluidas > 0
BEGIN
    DELETE TOP (5000) FROM stg_nfe_importacao_arquivo_morto
```

```

WHERE data_hora_expurgo < DATEADD(YEAR, -5, GETDATE());

SET @linhas_excluidas = @@ROWCOUNT;

-- Opcional: pequena pausa entre lotes, para reduzir contenção em ambiente de
produção
WAITFOR DELAY '00:00:00.5';
END;

```

@@ROWCOUNT retorna o número de linhas afetadas pelo **último** comando executado — aqui, usamos essa variável para controlar o laço: quando um lote não excluir mais nenhuma linha (**@@ROWCOUNT = 0**), significa que não há mais nada a excluir, e o laço termina.

4.8 Resumo da Sintaxe Completa do DELETE

```

[ WITH cte AS (...) ]
DELETE [ TOP (n) ] [ FROM ] { tabela | cte | alias }
[ OUTPUT clausula_output [ INTO tabela_destino ] ]
[ FROM origem_do_join [ JOIN ... ON ... ] ]
[ WHERE condicao ];

```

5. TRUNCATE TABLE × DELETE — Diferenças Fundamentais

Embora ambos removam dados de uma tabela, **TRUNCATE TABLE** e **DELETE FROM tabela** (sem **WHERE**) **não são equivalentes**. Um erro comum é tratá-los como intercambiáveis.

Aspecto	DELETE	TRUNCATE TABLE
Permite WHERE (remover parte dos dados)	Sim	Não — remove sempre todas as linhas
Registro em log de transações	Linha a linha (log completo)	Mínimo (apenas as páginas desalocadas)
Velocidade em tabelas grandes	Mais lento	Muito mais rápido

Aspecto	DELETE	TRUNCATE TABLE
Dispara <i>triggers</i> AFTER DELETE	Sim	Não
Reseta colunas IDENTITY	Não (o contador continua de onde parou)	Sim (volta à semente original, a menos que resetado manualmente)
Permissão necessária	DELETE na tabela	ALTER na tabela
Funciona se a tabela é referenciada por FOREIGN KEY de outra tabela com linhas	Sim (respeitando a integridade, linha a linha)	Não — falha imediatamente, mesmo que a tabela referenciadora esteja vazia (a menos que a FK seja removida ou desabilitada antes)
Pode ser revertido dentro de uma transação (ROLLBACK)	Sim	Sim (ambos são operações logadas o suficiente para rollback dentro da mesma transação)

Exemplo fiscal — quando usar **TRUNCATE:** limpar completamente uma tabela de staging (**stg_nfe_importacao**) entre cargas de teste, sem qualquer necessidade de preservar histórico ou disparar *triggers*:

```
TRUNCATE TABLE stg_nfe_importacao;
```

Exemplo fiscal — quando **TRUNCATE NÃO deve ser usado:** tentar limpar a tabela **nfe**, que é referenciada por **item_nfe**, **evento_nfe**, **duplicata_nfe** e outras — o comando falhará com erro, mesmo que a intenção fosse legítima, justamente porque o **TRUNCATE** não verifica linha a linha se há dependência, apenas recusa a operação inteira diante de qualquer **FOREIGN KEY** de referência.

```
TRUNCATE TABLE nfe;
-- Msg 4712: Cannot truncate table 'nfe' because it is being referenced by a
FOREIGN KEY constraint.
```

6. Interação com Integridade Referencial e Triggers

6.1 DELETE e as ações ON DELETE

Como estudado no módulo de Restrições e Integridade, o comportamento de um `DELETE` sobre uma tabela "pai" depende diretamente da cláusula `ON DELETE` definida na `FOREIGN KEY` da tabela "filha":

```
DELETE FROM nfe WHERE id_nfe = 1042;
```

- Se `item_nfe.fk_item_nfe` foi criada com `ON DELETE CASCADE`, os itens daquela NF-e são excluídos automaticamente junto.
- Se `evento_nfe.fk_evento_nfe` foi criada com `NO ACTION` (padrão), a exclusão da NF-e **falhará** caso existam eventos associados a ela — protegendo o histórico de eventos de ser perdido silenciosamente.

6.2 UPDATE e Triggers de Validação

Como visto no módulo de Restrições e Integridade, um `UPDATE` pode ser interceptado por uma *trigger* `AFTER UPDATE` que valide a transição de estado antes de permitir a alteração — por exemplo, a *trigger* `trg_impede_reabertura_nfe`, que bloqueia a mudança de `situacao_nfe` de `'CANCELADA'` para qualquer outro valor:

```
UPDATE nfe
SET situacao_nfe = 'AUTORIZADA'
WHERE id_nfe = 1042 AND situacao_nfe = 'CANCELADA';
-- Se a trigger trg_impede_reabertura_nfe estiver ativa, este comando é revertido
-- e retorna a mensagem definida no RAISERROR daquela trigger.
```

Implicação prática: ao escrever um `UPDATE` em massa (como os das seções 3.5 ou 3.9), é importante ter em mente **todas** as *triggers* ativas na tabela-alvo — uma única linha do lote que viole uma regra de trigger pode fazer o `ROLLBACK` de todo o lote, dependendo de como a trigger foi escrita (lembrando que, por padrão em T-SQL, uma trigger dispara **uma única vez por comando**, recebendo todas as linhas afetadas de uma vez nas tabelas `inserted/deleted` — não uma vez por linha).

7. Segurança Operacional: Evitando Desastres em Produção

7.1 A regra de ouro: nunca escrever WHERE por último

O erro mais comum — e mais destrutivo — em **UPDATE/DELETE** é **esquecer a cláusula WHERE**, afetando a tabela inteira.

```
-- ⚠ PERIGO: sem WHERE, cancela TODAS as NF-e da base
UPDATE nfe
SET situacao_nfe = 'CANCELADA';
```

Prática recomendada: sempre escrever primeiro um **SELECT** com a mesma condição pretendida, **conferir visualmente as linhas retornadas**, e só então transformar o **SELECT** em **UPDATE/DELETE**:

```
-- Passo 1: sempre comece assim, e revise o resultado
SELECT * FROM nfe WHERE id_nfe = 1042;

-- Passo 2: só depois de confirmar visualmente, transforme em UPDATE/DELETE
UPDATE nfe SET situacao_nfe = 'CANCELADA' WHERE id_nfe = 1042;
```

7.2 Envolvendo em uma Transação Explícita, com Verificação Antes do COMMIT

Para operações de maior risco (alterações em massa, exclusões em tabelas críticas), é recomendável envolver o comando em uma transação explícita, verificar **@@ROWCOUNT** (ou consultar o resultado) **antes** de confirmar, e só então decidir entre **COMMIT** ou **ROLLBACK**.

```
BEGIN TRANSACTION;

UPDATE auto_infracao
SET valor_multa_aplicado = valor_multa_aplicado * 1.10
WHERE situacao = 'LAVRADO';
```

```

PRINT 'Linhas afetadas: ' + CAST(@@ROWCOUNT AS VARCHAR(10));

-- Neste ponto, o instrutor/DBA verifica manualmente se o número de linhas faz
-- sentido
-- (por exemplo, comparando com uma contagem prévia esperada) antes de decidir:

-- COMMIT TRANSACTION; -- confirma a alteração
-- ROLLBACK TRANSACTION; -- desfaz tudo, caso o número de linhas pareça errado

```

Enquanto a transação não é finalizada com **COMMIT** ou **ROLLBACK**, **nenhuma outra sessão consegue enxergar a alteração**, e ela pode ser completamente desfeita. Esse é o principal mecanismo de segurança contra erros operacionais em comandos de alto risco.

7.3 TRY...CATCH para Scripts de Produção

Para scripts automatizados (não interativos), o padrão **TRY...CATCH** combinado com transação garante que qualquer erro durante a execução resulte em **ROLLBACK** automático, evitando que a base fique em um estado parcialmente alterado.

```

BEGIN TRY
    BEGIN TRANSACTION;

    UPDATE auto_infracao
    SET valor_multa_aplicado = valor_multa_aplicado * 1.10
    WHERE situacao = 'LAVRADO';

    DELETE FROM stg_nfe_importacao
    WHERE chave_acesso IN (SELECT chave_acesso FROM nfe_importada);

    COMMIT TRANSACTION;
END TRY
BEGIN CATCH
    IF @@TRANCOUNT > 0
        ROLLBACK TRANSACTION;

    PRINT 'Erro durante a operação: ' + ERROR_MESSAGE();
END CATCH;

```

7.4 Uso de SET XACT_ABORT ON

Em scripts que combinam múltiplos comandos DML dentro de uma transação, **SET XACT_ABORT ON** garante que **qualquer** erro de execução (mesmo os que, por padrão, não encerrariam automaticamente a transação) provoque o cancelamento e o

ROLLBACK imediato de toda a transação — uma proteção adicional recomendada para scripts de produção.

```
SET XACT_ABORT ON;

BEGIN TRANSACTION;
    UPDATE auto_infracao SET valor_multa_aplicado = valor_multa_aplicado * 1.10
WHERE situacao = 'LAVRADO';
COMMIT TRANSACTION;
```

8. Roteiro Prático Completo

Objetivo: aplicar, em sequência, as principais variações de **UPDATE** e **DELETE** estudadas neste módulo sobre o banco **curso_integridade_fiscal**.

```
USE curso_integridade_fiscal;
GO

-- 1. UPDATE básico
UPDATE nfe
SET situacao_nfe = 'CANCELADA', data_cancelamento = SYSDATETIME()
WHERE id_nfe = 1;
GO

-- 2. UPDATE ... FROM (join) com reajuste de multas
UPDATE a
SET a.valor_multa_aplicado = t.valor_multa_base
FROM auto_infracao a
JOIN tipo_infracao t ON t.id_tipo_infracao = a.id_tipo_infracao
WHERE a.situacao = 'LAVRADO';
GO

-- 3. UPDATE com OUTPUT para auditoria
CREATE TABLE auditoria_reajuste_multa (
    id_auditoria          BIGINT IDENTITY(1,1) NOT NULL,
    id_auto_infracao      INT                NOT NULL,
    valor_anterior        NUMERIC(15,2) NOT NULL,
    valor_novo            NUMERIC(15,2) NOT NULL,
    data_hora_alteracao   DATETIME2         NOT NULL DEFAULT (SYSDATETIME()),
    CONSTRAINT pk_auditoria_reajuste_multa PRIMARY KEY (id_auditoria)
);
GO

UPDATE a
SET a.valor_multa_aplicado = a.valor_multa_aplicado * 1.05
OUTPUT deleted.id_auto_infracao, deleted.valor_multa_aplicado,
inserted.valor_multa_aplicado
INTO auditoria_reajuste_multa (id_auto_infracao, valor_anterior, valor_novo)
```

```

FROM auto_infracao a
WHERE a.situacao = 'LAVRADO';
GO

-- 4. DELETE simples
DELETE FROM duplicata_nfe WHERE id_duplicata = 1;
GO

-- 5. DELETE ... FROM (join)
DELETE i
FROM item_nfe i
JOIN nfe n ON n.id_nfe = i.id_nfe
WHERE n.situacao_nfe = 'DENEGADA';
GO

-- 6. DELETE em lote com WHILE
DECLARE @linhas_excluidas INT = 1;
WHILE @linhas_excluidas > 0
BEGIN
    DELETE TOP (100) FROM stg_nfe_importacao
    WHERE data_hora_carga < DATEADD(DAY, -90, GETDATE());
    SET @linhas_excluidas = @@ROWCOUNT;
END;
GO

-- 7. Transação segura com verificação antes do COMMIT
BEGIN TRANSACTION;

UPDATE contribuinte
SET situacao_cadastral = 'SUSPENSO'
WHERE id_contribuinte IN (
    SELECT id_contribuinte FROM contribuinte WHERE situacao_cadastral = 'ATIVO'
    AND NOT EXISTS (SELECT 1 FROM nfe n WHERE n.id_contribuinte_emitente =
contribuinte.id_contribuinte)
);

SELECT @@ROWCOUNT AS linhas_afetadas;
-- Revisar o número antes de decidir:
-- COMMIT TRANSACTION;
-- ROLLBACK TRANSACTION;

```

9. Exercícios Práticos

Exercício 1 — UPDATE básico e múltiplas colunas

Escreva um **UPDATE** que marque a ordem de serviço de fiscalização **id_ordem_servico = 5** como **'CONCLUIDA'**, preenchendo também **data_encerramento** com a data atual, em um único comando.

Exercício 2 — UPDATE com expressão

A SEFAZ-PB decidiu conceder um desconto de 20% no valor de multas aplicadas há mais de 5 anos e ainda não quitadas, como incentivo à regularização. Escreva o **UPDATE** correspondente, com a condição de tempo adequada.

Exercício 3 — UPDATE ... FROM com múltiplas tabelas

Escreva um **UPDATE** que, para cada **ordem_servico_fiscalizacao** com **status = 'EM_ANDAMENTO'** há mais de 180 dias, marque **status = 'CONCLUIDA'** — mas apenas para as ordens de serviço que já possuem **pelo menos um** **auto_infracao** lavrado (use **EXISTS** ou **JOIN**, à sua escolha, e justifique a escolha).

Exercício 4 — UPDATE com OUTPUT

Ao aplicar o Exercício 3, capture, por meio de **OUTPUT**, o **id_ordem_servico** e a data em que cada uma foi automaticamente concluída, gravando essa informação em uma nova tabela **auditoria_encerramento_os**.

Exercício 5 — DELETE com subconsulta

Escreite um **DELETE** que remova, da tabela **evento_nfe**, todos os eventos do tipo **'MANIFESTACAO_DESTINATARIO'** associados a NF-e que estão **'CANCELADA'** — pressupondo que, uma vez cancelada a nota, esse tipo específico de evento perde o sentido (regra hipotética para fins do exercício).

Exercício 6 — DELETE ... FROM com JOIN

Escreva um **DELETE** que remova, de **cte_nfe_transportada**, todos os vínculos de CT-e com NF-e que estejam **'DENEGADA'**.

Exercício 7 — TRUNCATE vs DELETE

Explique, sem executar, o que aconteceria se você tentasse `TRUNCATE TABLE contribuinte` no banco `curso_integridade_fiscal`, considerando as tabelas e chaves estrangeiras já criadas nos módulos anteriores. Qual seria a mensagem de erro esperada, e por quê?

Exercício 8 — Exclusão em lote

Escreva um script de exclusão em lote (padrão `WHILE + TOP + @@ROWCOUNT`) que remova, em lotes de 200, todos os registros de `nfe_importacao_rejeitada` com mais de 1 ano.

Exercício 9 (Desafio) — Planejando uma operação de alto risco

A Secretaria pediu que todos os `auto_infracao` do `tipo_infracao` com `id_tipo_infracao = 7` sejam anulados (`situacao = 'ANULADO'`), pois aquela infração foi revogada retroativamente por decisão judicial. Escreva o **script completo e seguro** dessa operação, incluindo:

- A consulta `SELECT` prévia de verificação.
- A transação explícita com checagem de `@@ROWCOUNT` antes do `COMMIT`.
- A captura, via `OUTPUT`, de todos os autos afetados, para fins de notificação posterior aos contribuintes.

10. Glossário

Termo	Definição
DML (Data Manipulation Language)	Subconjunto de comandos SQL que manipulam dados (<code>SELECT</code> , <code>INSERT</code> , <code>UPDATE</code> , <code>DELETE</code>)
Cláusula FROM (em UPDATE/DELETE)	Em T-SQL, introduz um <code>JOIN</code> com outra tabela para orientar quais linhas atualizar/excluir
OUTPUT	Cláusula que captura os valores antes (<code>deleted</code>) e/ou depois (<code>inserted</code>) de um <code>UPDATE</code> , <code>DELETE</code> ou <code>INSERT</code>

Termo	Definição
Batched DELETE (exclusão em lote)	Padrão de excluir grandes volumes em pequenos lotes sucessivos, para reduzir impacto no log de transações e nos bloqueios
@@ROWCOUNT	Variável de sistema que retorna o número de linhas afetadas pelo último comando
TRUNCATE TABLE	Comando DDL que remove todas as linhas de uma tabela de forma minimamente registrada, sem WHERE e sem disparar triggers AFTER DELETE
XACT_ABORT	Configuração de sessão que força o ROLLBACK automático de toda a transação diante de qualquer erro em tempo de execução

11. Referências

- Documentação oficial da Microsoft — "UPDATE (Transact-SQL)":
<https://learn.microsoft.com/sql/t-sql/queries/update-transact-sql>
- Documentação oficial da Microsoft — "DELETE (Transact-SQL)":
<https://learn.microsoft.com/sql/t-sql/statements/delete-transact-sql>
- Documentação oficial da Microsoft — "TRUNCATE TABLE (Transact-SQL)":
<https://learn.microsoft.com/sql/t-sql/statements/truncate-table-transact-sql>
- Documentação oficial da Microsoft — "OUTPUT Clause (Transact-SQL)":
<https://learn.microsoft.com/sql/t-sql/queries/output-clause-transact-sql>
- Documentação oficial da Microsoft — "SET XACT_ABORT (Transact-SQL)":
<https://learn.microsoft.com/sql/t-sql/statements/set-xact-abort-transact-sql>
- ELMASRI, R.; NAVATHE, S. *Sistemas de Banco de Dados*. 7ª ed. Pearson.

Material elaborado para uso interno no curso de Banco de Dados Relacionais — Módulo UPDATE e DELETE — SEFAZ-PB.